Viva Cheat Sheet - DevOps, DevSecOps, And Security

Main Message To Say

EventHub is a microservice-based application deployed using modern DevOps practices. Each service is containerized with Docker, built and checked through GitHub Actions, published as a container image, scanned with SonarCloud for DevSecOps, and deployed to Azure Container Apps. Security is handled through JWT authentication, role-based access, environment secrets, and internal-only microservice exposure behind a public API gateway.

DevOps Points

Version Control

What to show:

- GitHub repository
- project folders for each service
- `.github/workflows`

What to say:

The source code is maintained in GitHub. Each microservice has its own folder, Dockerfile, routes, controllers, models, and service-specific configuration. GitHub Actions is used to automate checks, image publishing, SonarCloud scanning, and Azure deployment.

CI Pipeline

What to show:

- `.github/workflows/ci.yml`

What to say:

The CI pipeline runs automatically when code is pushed or when pull requests are created. It installs dependencies and runs tests to catch errors before deployment.

Possible viva question:

Why do we need CI?

Answer:

CI helps detect broken code early. Instead of manually checking everything, every push can automatically run tests and validation, which improves reliability and team collaboration.

Docker Containerization

What to show:

- `services/auth-service/Dockerfile`

- `services/event-service/Dockerfile`
- `services/booking-service/Dockerfile`
- `services/notification-service/Dockerfile`
- `gateway/Dockerfile`
- `docker-compose.yml`

What to say:

Each microservice is containerized independently. This means each service can be built, shipped, and deployed consistently across local and cloud environments.

Possible viva question:

Why use Docker?

Answer:

Docker packages the application with its runtime dependencies. It reduces environment differences between developer machines and cloud deployment.

Container Registry

What to show:

- `.github/workflows/docker-publish.yml`
- GitHub Packages / GHCR images if available

What to say:

The Docker images are published to GitHub Container Registry. Azure Container Apps consumes those images during deployment.

Possible viva question:

Why use a container registry?

Answer:

A registry stores versioned container images. It acts as the delivery point between CI/CD and the cloud runtime.

Cloud Deployment

What to show:

- `.github/workflows/deploy-azure.yml`
- Azure Portal Container Apps
- running services in `eventhub-rg`

Command to show:

```
az containerapp list --query "[].{name:name, resourceGroup:resourceGroup,
fqdn:properties.configuration.ingress.fqdn, status:properties.runningStatus}" -o
table
```

What to say:

The backend is deployed on Azure Container Apps. The gateway has external ingress, while the individual microservices use internal ingress. This keeps the system accessible through one public endpoint and reduces direct exposure of internal services.

Possible viva question:

Why Azure Container Apps?

Answer:

Azure Container Apps is a managed container platform. It lets us run containerized microservices without managing Kubernetes infrastructure directly.

DevSecOps Points

SonarCloud Static Analysis

What to show:

- `.github/workflows/sonarcloud.yml`
- `sonar/auth-service.properties`
- `sonar/event-service.properties`
- `sonar/booking-service.properties`
- `sonar/notification-service.properties`
- SonarCloud dashboard

What to say:

SonarCloud is integrated into GitHub Actions. The workflow scans each microservice separately, so code quality and security issues can be tracked per service.

The scanned services are:

- EventHub Auth Service
- EventHub Event Service
- EventHub Booking Service
- EventHub Notification Service

Possible viva question:

What does SonarCloud check?

Answer:

SonarCloud checks bugs, vulnerabilities, security hotspots, code smells, duplication, maintainability, reliability, and coverage information.

Why Separate SonarCloud Projects?

Answer:

Because the system follows microservice architecture. Each service is owned independently, so scanning them separately makes it easier to identify which service has quality or security issues.

Quality Gate

What to say:

The quality gate gives a pass/fail result based on configured quality rules. It helps decide whether code is safe enough to merge or deploy.

Possible viva question:

What if SonarCloud finds a vulnerability?

Answer:

We inspect the issue, understand the risk, fix the affected code, and rerun the pipeline. The goal is to prevent insecure code from being accepted silently.

Security Points

API Gateway Security

What to show:

- `gateway/src/app.js`
- `gateway/src/routes/index.js`
- Azure gateway URL

What to say:

The frontend communicates only with the API gateway. Internal microservices are not directly exposed to the internet. This follows the least-exposure principle.

Possible viva question:

Why not expose every service publicly?

Answer:

Exposing every service increases the attack surface. A gateway gives one controlled public entry point and keeps internal services private.

JWT Authentication

What to show:

- `services/auth-service/src/controllers/authController.js`
- `services/auth-service/src/middleware/auth.js`

What to say:

Users log in through the auth service and receive a JWT token. Protected endpoints verify this token before allowing access.

Possible viva question:

Why use JWT?

Answer:

JWT supports stateless authentication. Services can verify the token without storing server-side sessions.

Role-Based Access Control

Roles:

- `customer`
- `organizer`
- `admin`

What to say:

Different roles have different permissions. Customers can request bookings, organizers can manage events and approvals, and admins can oversee system activity.

Possible viva question:

How do you prevent a customer from accessing organizer functions?

Answer:

Protected middleware checks the authenticated user's role before allowing restricted actions.

Secrets And Environment Variables

What to show:

- GitHub Actions secrets list names only, not values
- Azure Container Apps environment variables names only

What to say:

Sensitive values such as MongoDB URIs, JWT secrets, service-to-service secrets, and registry tokens are not hardcoded into the workflow. They are configured through GitHub Secrets and environment variables.

Important:

Do not open `.env` files during viva if they contain real credentials.

Possible viva question:

Why use GitHub Secrets?

Answer:

Secrets keep sensitive values out of source code while still allowing CI/CD workflows to use them securely.

Internal Service Communication

What to say:

The gateway routes frontend requests to services. For backend workflows, booking-service communicates with event-service, notification-service, and auth-service using internal service URLs.

Possible viva question:

What is your strongest inter-service communication example?

Answer:

The booking flow. When a customer requests a booking, booking-service calls event-service to reserve seats, notification-service to create alerts, and auth-service to find admin users for admin notifications.

Basic Express Security

What to show:

- `helmet` usage in service or gateway app files
- `cors` usage
- validation middleware

What to say:

Helmet helps set safer HTTP headers. CORS controls browser access. Validation middleware helps reject invalid request payloads before business logic runs.

10-Minute Viva Demo Flow

1. Show architecture diagram.
2. Open Vercel frontend.
3. Show Azure gateway health:

```
curl https://gateway.mangocoast-efcefaea.southeastasia.azurecontainerapps.io
```

4. Show Azure Container Apps list.
5. Run the app flow:
    - login/register
    - create event
    - request booking
    - approve/reject booking
    - show notification

6. Explain service integration during booking.
7. Show GitHub Actions workflows.
8. Show Dockerfiles and GHCR/container image workflow.
9. Show SonarCloud project dashboard.
10. Explain security summary.

High-Value Questions And Short Answers

What DevOps practices did you implement?

We used GitHub version control, GitHub Actions CI/CD, Docker containerization, GHCR image publishing, Azure Container Apps deployment, and automated SonarCloud scanning.

What DevSecOps practice did you implement?

We integrated SonarCloud into GitHub Actions to scan code for bugs, vulnerabilities, security hotspots, code smells, duplication, and maintainability issues.

How is the application secured?

We use JWT authentication, role-based authorization, protected routes, environment secrets, internal-only microservice exposure, API gateway routing, and static analysis through SonarCloud.

What is the cloud capability used?

Azure Container Apps is used to host the backend containers. It provides managed container hosting and supports external/internal ingress for microservices.

What is least privilege in your project?

Only the gateway is public. Internal services are reachable only through the gateway or internal service communication. Secrets are stored in GitHub Secrets and environment variables instead of source code.

What would you improve if this became production?

I would add stronger service-to-service authentication, rate limiting, centralized logging, monitoring alerts, HTTPS-only policy enforcement, managed secret storage such as Azure Key Vault, and message queues for more reliable async workflows.

Things To Avoid During Viva

- Do not open `.env` files with passwords.
- Do not say every microservice is publicly exposed. Say the gateway is public and services are internal for security.
- Do not focus only on UI. Keep bringing the answer back to DevOps, DevSecOps, cloud deployment, and security.
- Do not claim Kubernetes if you used Azure Container Apps.
- Do not say SonarCloud fixes issues automatically. It detects and reports issues; developers fix them.